

MIPS Processor Design Recap

`mano@cs.auckland.ac.nz`

MIPS Introduction

- MIPS is a RISC (reduced instruction set computer) processor.
- We will briefly look at some of the principles of processor design in the light of improving performance and using concurrency.
- We will start with the very basics.

Computer Organization and Design – The
Hardware Software Interface RISC-V
edition, 2nd edition (2020)

Instruction Set Architecture (ISA)

- This is the programmers view of the processor. It defines the machine language exposed by the processor.
 - Machine language is just a bunch of 0s and 1s. Hard for any of us to speak.
 - An assembly language is a human-readable form of the machine language.
 - So an ISA mostly uses assembly language to denote the machine language.
 - An assembler is a program that translates assembly language to machine language.
 - Compilers translate high-level languages (e.g., C++) to assembly language.
- Strictly speaking an ISA is the compiler's view of the processor.

Storage space

寄存器.

- A processor has ~~two~~ sets of storage space: a bank of registers and a linear array of memory.
- Memory is organized as bytes and each byte is individually addressable.
 - Words are the smallest units of data we get out of/into memory
 - A word, for example, could be 4 bytes (32 bits) or 8 bytes (64 bits)

0	8 bits of data
1	8 bits of data
2	8 bits of data
...	8 bits of data
...	8 bits of data

MIPS Storage Space

- For MIPS, a word is 32 bits or 4 bytes.
 - 2^{32} bytes with byte addresses from 0 to $2^{32}-1$
 - 2^{30} words with byte addresses 0, 4, 8, ... $2^{32}-4$
- Registers hold 32 bits of data, and there are 32 registers

0	8 bits of data
1	8 bits of data
2	8 bits of data
...	8 bits of data
...	8 bits of data

0	32 bits of data
4	32 bits of data
8	32 bits of data
12	32 bits of data
...	32 bits of data

MIPS Registers – Assembler Conventions

0	zero		16	s0	被调用函数 Callee saves
1	at	汇编程序 Reserved for assembler	...	...	(caller can clobber)
2	v0	Expression evaluation	23	s7	
3	v1	Function results	24	t8	Temp (cont.)
4	a0	Function arguments	25	t9	
5	a1		26	k0	Reserved for OS kernel
6	a2		27	k1	
7	a3		28	gp	Global pointer
8	t0	Temp (caller saves)	29	sp	Stack pointer
...	...	(callee can clobber)	30	fp	Frame pointer
15	t7	function	31	ra	Return address

U-31号寄存器说明

编号	名称	作用	说明
0	zero	恒为0	读总是返回0，写无效
1	at	保留给汇编器	assembler temporary, 汇编器内部使用
2-3	v0-v1	表达式/函数返回值	用于返回函数结果
4-7	a0-a3	函数参数	前4个参数通过寄存器传递
8-15	t0-t7	临时变量 (caller保存)	调用者在调用函数前需保存
16-23	s0-s7	保存变量 (callee保存)	被调用者保存和恢复这些值
24-25	t8-t9	临时变量 (继续使用)	与t0-t7一样, caller需保存
26-27	k0-k1	OS内核保留	仅操作系统使用
28	gp	全局指针	指向全局数据区域
29	sp	栈指针	指向当前栈顶
30	fp	帧指针	用于访问当前栈帧中的变量 (可选)
31	ra	返回地址	存放函数调用的返回地址

MIPS instructions – A selection

Instruction	Meaning	Type
add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	R
sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	R
lw \$s1, 100(\$s2)	$\$s1 = \text{Memory}[\$s2+100]$	I
sw \$s1, 100(\$s2)	$\text{Memory}[\$s2+100] = \$s1$	I
bne \$s4,\$s5,Label	Next instr. is at Label if $\$s4 \neq \$s5$	I
beq \$s4,\$s5,Label	Next instr. is at Label if $\$s4 = \$s5$	I
j Label	Next instr. is at Label	J

Instruction Types

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R	op	rs	rt	rd	shamt	func
I	op	rs	rt	16 bit address		
J	op	26 bit address				

For branch instructions, the address is an *offset* from the current program counter (PC).

Question: how can we do a longer branch than permissible by the 16 bit offset? Answer:

beq \$t0, \$t1, far

could be implemented as

bneq \$t0, \$t1, near

j far

near: ...

Assembler Macros

- These are convenience “instructions” provided by the assembler. Assembler translates them to real instruction during assembly.
 - E.g. the macro move \$t0, \$1 will be translated to add \$t0, \$t1, \$zero.

$t_0 = t_1 + 0$

Question – Assembler Macros

- Question: MIPS has an slt (set if less than) instruction: slt \$t1, \$s1, \$s2 will set \$t1 to one if \$s1 is less than \$s2, otherwise it clear \$t1 (i.e., set it to 0). How would you implement a new blt (branch less than) instruction as an assembler macro using slt and bne?
- Answer: blt \$s1, \$s2, label could be implemented as:
 slt \$t0, \$s1, \$s2
 bne \$t0, \$zero, label

Questions: (a) do you see any potential for problems in the implementation? (b) can we consider using beq instead of bne?

MIPS Arithmetic

Instruction	Example	Meaning
add	add \$1, \$2, \$3	$\$1 = \$2 + \$3$
subtract	sub \$1, \$2, \$3	$\$1 = \$2 - \$3$
add immediate	add \$1, \$2, 100	$\$1 = \$2 + 100$
add unsigned	addu \$1, \$2, \$3	$\$1 = \$2 + \$3$
subtract unsigned	subu \$1, \$2, \$3	$\$1 = \$2 - \$3$
add immediate unsigned	addiu \$1, \$2, 100	$\$1 = \$2 + 100$
multiply	mult \$2, \$3	Hi, Lo = $\$2 * \3
multiply unsigned	multu \$2, \$3	Hi, Lo = $\$2 * \3
divide	div \$2, \$3	Lo = $\$2 / \3 ; Hi = $\$2 \bmod \3
divide unsigned	divu \$2, \$3	Lo = $\$2 / \3 ; Hi = $\$2 \bmod \3
move from hi	mfhi \$1	$\$1 = \text{Hi}$
move from lo	mflo \$1	$\$1 = \text{Lo}$

MIPS Logical

Instruction	Example	Meaning
and	and \$1, \$2, \$3	\$1 = \$2 and \$3
or	or \$1, \$2, \$3	\$1 = \$2 or \$3
and immediate	andi \$1, \$2, 100	\$1 = \$2 and 100
or immediate	ori \$1, \$2, 100	\$1 = \$2 or 100
shift left logical	sll \$1, \$2, 10	\$1 = \$2 << 10
shift right logical	srl \$1, \$2, 10	\$1 = \$2 >> 10

MIPS Data Transfer

Instruction	Example	Meaning
load word	lw \$1, 100(\$2)	$\$1 = \text{Memory}[\$2+100]$
store word	sw \$1, 100(\$2)	$\text{Memory}[\$2+100] = \1
load upper immediate	lui \$1, 100	$\$1 = 100 * 2^{16}$

MIPS Control Transfer

Instruction	Example	Meaning
branch equal	beq \$1, \$2, L1	if (\$1 == \$2) go to L1
branch not equal	bne \$1, \$2, L1	if (\$1 != \$2) go to L1
set less than	slt \$1, \$2, \$3	\$1 = (\$2<\$3) ? 1 : 0
set less than immediate	slti \$1, \$2, 100	\$1 = (\$2<100) ? 1 : 0
set less than unsigned	sltu \$1, \$2, \$3	\$1 = (\$2<\$3) ? 1 : 0
slt immediate unsigned	sltiu \$1, \$2, 100	\$1 = (\$2<100) ? 1 : 0
jump	j 10000	go to 10000
jump register	j \$31	go to \$31
jump and link	jal 10000	\$31=PC+4; go to 10000

word length

Example: How to speedup Loops

```
while ( i < 10 ) {  
    sum += array[i];  
    ++i;  
}
```

```
for ( i = 0; i < 10; ++i ) {  
    sum += array[i];  
}
```

end..

```
addiu $t9, $a0, 40  
Loop: beq $a0, $t9, Exit  
      lw $t0, 0($a0)  
      add $s0, $s0, $t0  
      addiu $a0, $a0, 4  
      j Loop  
Exit: ...
```

word length.
↑
t4 : - - - -> 40

start element

$a0 = t9$, exit.

Each loop iteration has 2 branch instructions!

Example: How to speedup Loops

```
        addiu $t9, $a0, 40
Loop:   beq $a0, $t9, Exit
        lw $t0, 0($a0)
        add $s0, $s0, $t0
        addiu $a0, $a0, 4
        j Loop
Exit:   ...
```

```
        addiu $t9, $a0, 40
        beq $a0, $t9, Exit
Loop: lw $t0, 0($a0)
        add $s0, $s0, $t0
        addiu $a0, $a0, 4
        bne $a0, $t9, Loop
Exit:   ...
```

```
for (i = 0; i < 10; ++i) {
    sum += array[i];
}
```

Example: How to speedup Loops

```
    addiu $t9, $a0, 40  
    beq $a0, $t9, Exit  
Loop: lw $t0, 0($a0)  
      add $s0, $s0, $t0  
      addiu $a0, $a0, 4  
      bne $a0, $t9, Loop  
Exit: ...
```

```
    addiu $t9, $a0, 40  
  
Loop: lw $t0, 0($a0)  
      add $s0, $s0, $t0  
      addiu $a0, $a0, 4  
      bne $a0, $t9, Loop  
Exit: ...
```

```
for (i = 0; i < 10; ++i) {  
    sum += array[i];  
}
```

Example: Loop Unrolling

```
for (i = 0; i < 10; ++i ) {  
    sum += array[i];  
}
```

```
for (i = 0; i < 10; i +=1) {  
    sum += array[i];  
    ++i;  
    sum += array[i];  
}
```

```
addiu $t9, $a0, 40
```

```
Loop: lw $t0, 0($a0)  
      add $s0, $s0, $t0  
      addiu $a0, $a0, 4  
      lw $t0, 0($a0)  
      add $s0, $s0, $t0  
      addiu $a0, $a0, 4  
      bne $a0, $t9, Loop
```

```
Exit:  ...
```

4 instructions per iteration has now become 7 instructions per 2 iterations

Example: Loop Unrolling

```
for (i = 0; i < 10; ++i ) {  
    sum += array[i];  
}
```

```
for (i = 0; i < 10; i +=1) {  
    sum += array[i];  
    ++i;  
    sum += array[i];  
}
```

```
addiu $t9, $a0, 40
```

```
Loop: lw $t0, 0($a0)
```

```
add $s0, $s0, $t0
```

```
lw $t0, 4($a0)
```

```
add $s0, $s0, $t0
```

```
addiu $a0, $a0, 8
```

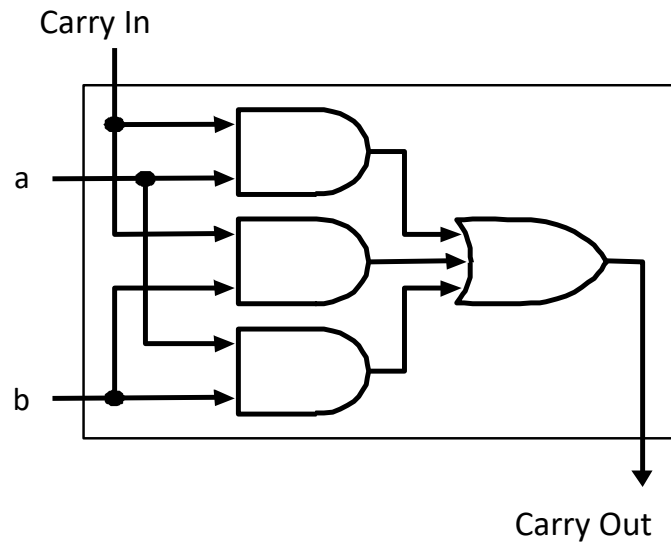
```
bne $a0, $t9, Loop
```

```
Exit:  ...
```

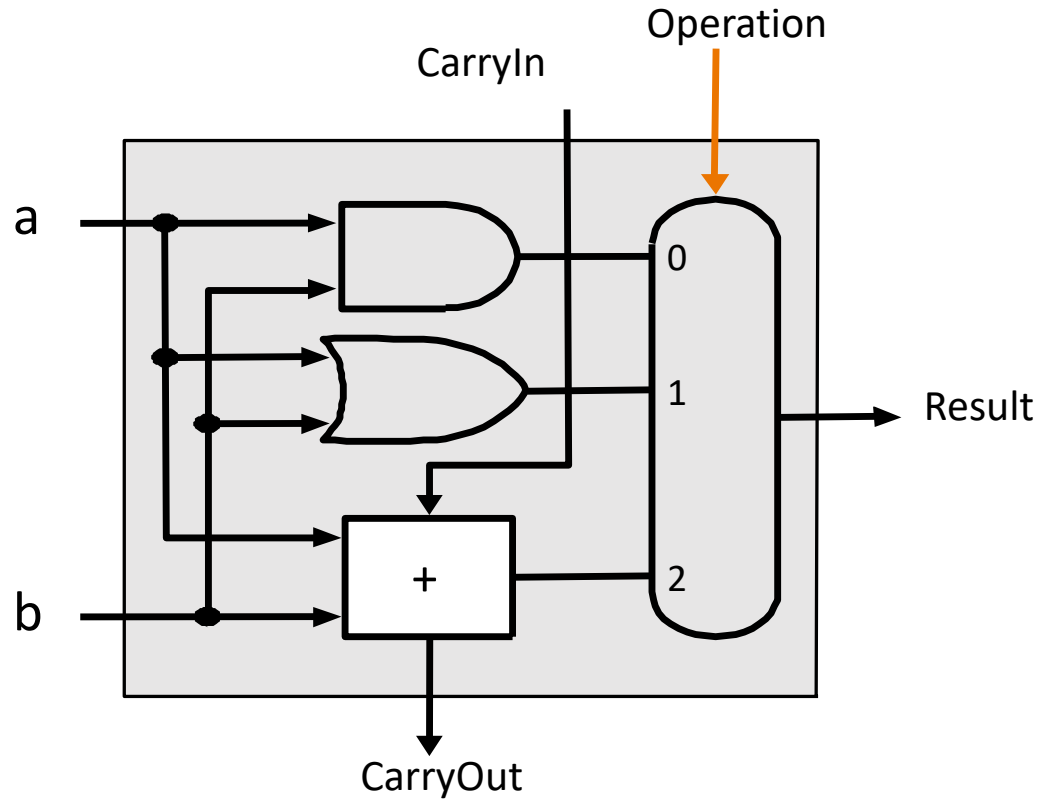
We now have 6 instructions per 2 iterations

Hardware bits

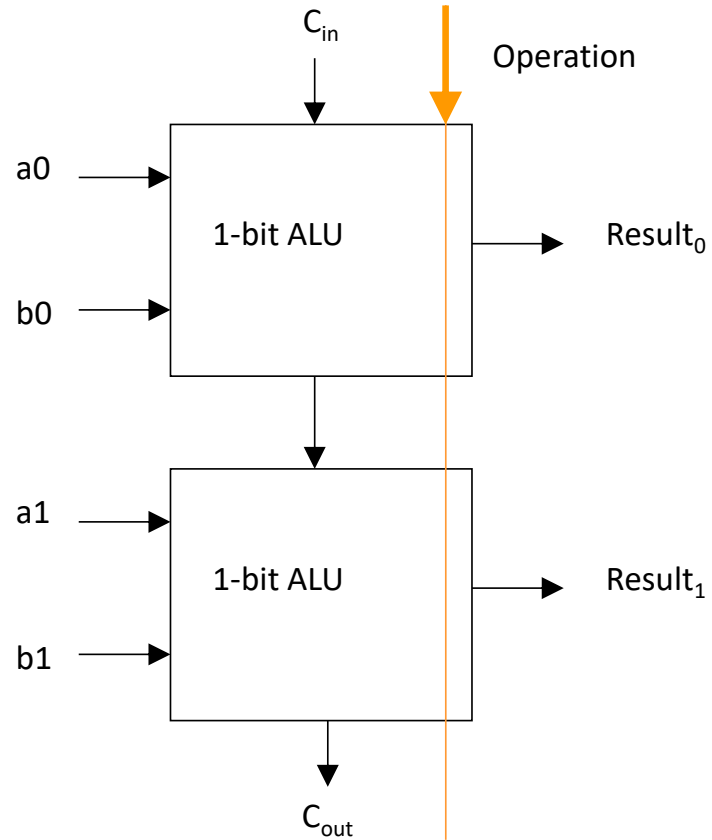
A Single Bit Adder



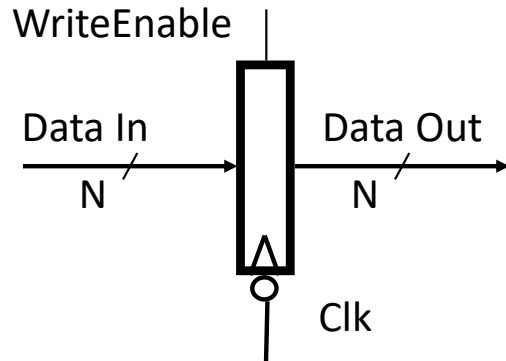
A Single Bit ALU



Multi Bit ALU



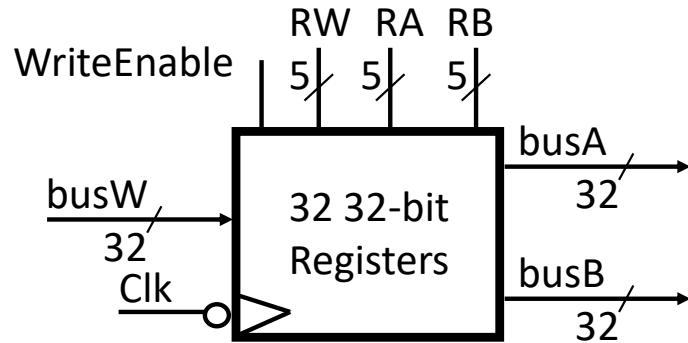
Register



Similar to the D Flip Flop except: N-bit input and output; and WriteEnable input

Combinational logic (or *combinatorial* logic) is a logic circuit whose output depends only on its current input. In contrast, in a *sequential* logic, the output will depend not only on the current input but also some past inputs. Sequential logic circuits have memory. A register here is a sequential logic circuit. The content of the register can be read anytime, but the register can only be written at the clock's tick only when WriteEnable is set (to 1).

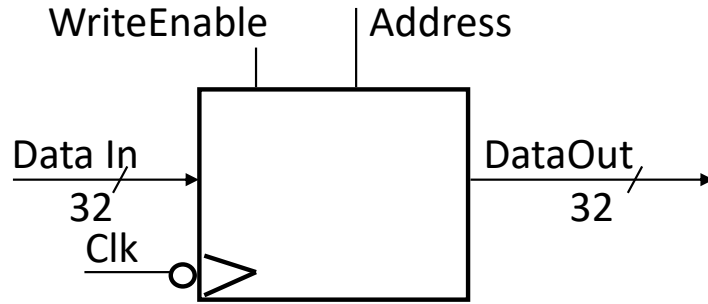
Register File



A register file is simply a set of registers. In case of MIPS, there are 32 registers in the register file.

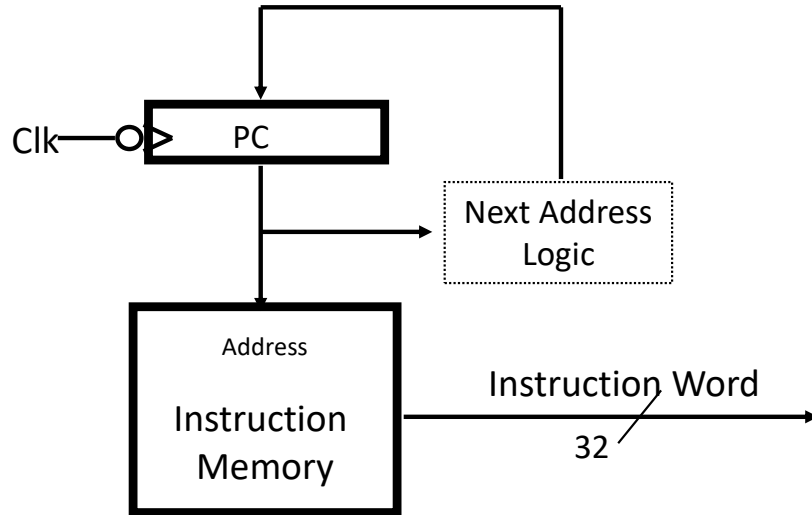
BusA and busB always pick up the content of the registers RA and RB (irrespective of the clock). Register RW is written to with the data on busW at the clock's tick only if WriteEnable is set.

Memory



The memory is idealized for simplicity. The CLK input is a factor only during write operation. During read operation, the memory behaves as a combinational logic block: DataOut becomes valid after access time.

Instruction Fetch Unit



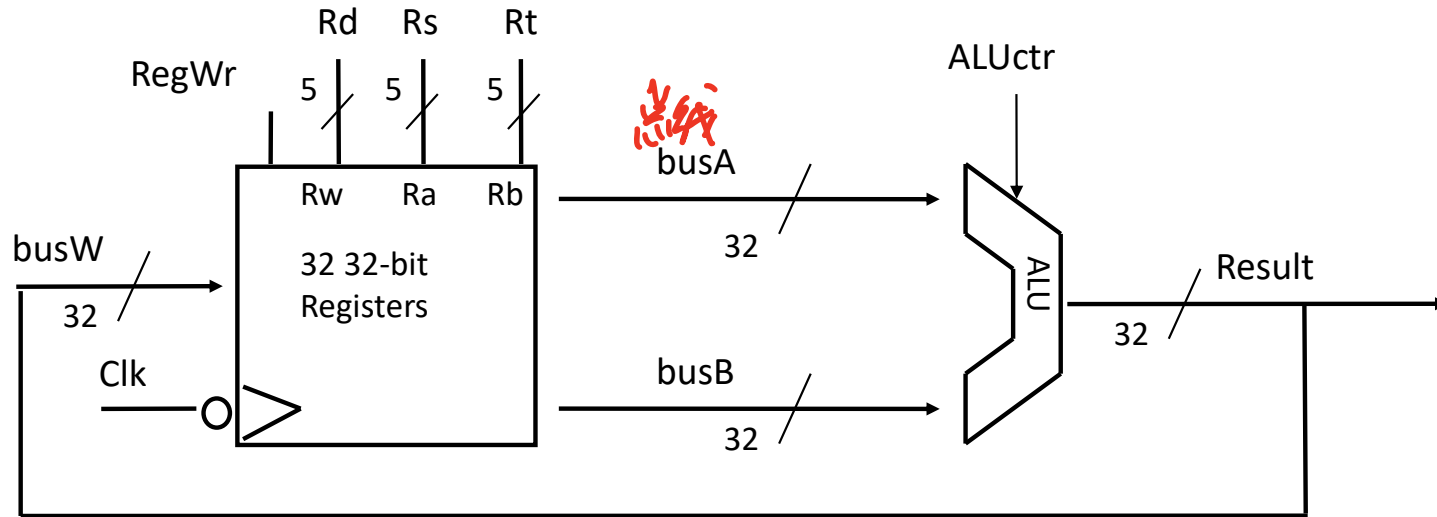
Fetch the Instruction: $\text{InstructionMemory}[\text{PC}]$

Update the program counter:

Sequential Code: $\text{PC} = \text{PC} + 4$

Branch and Jump: $\text{PC} = \text{"something else"}$

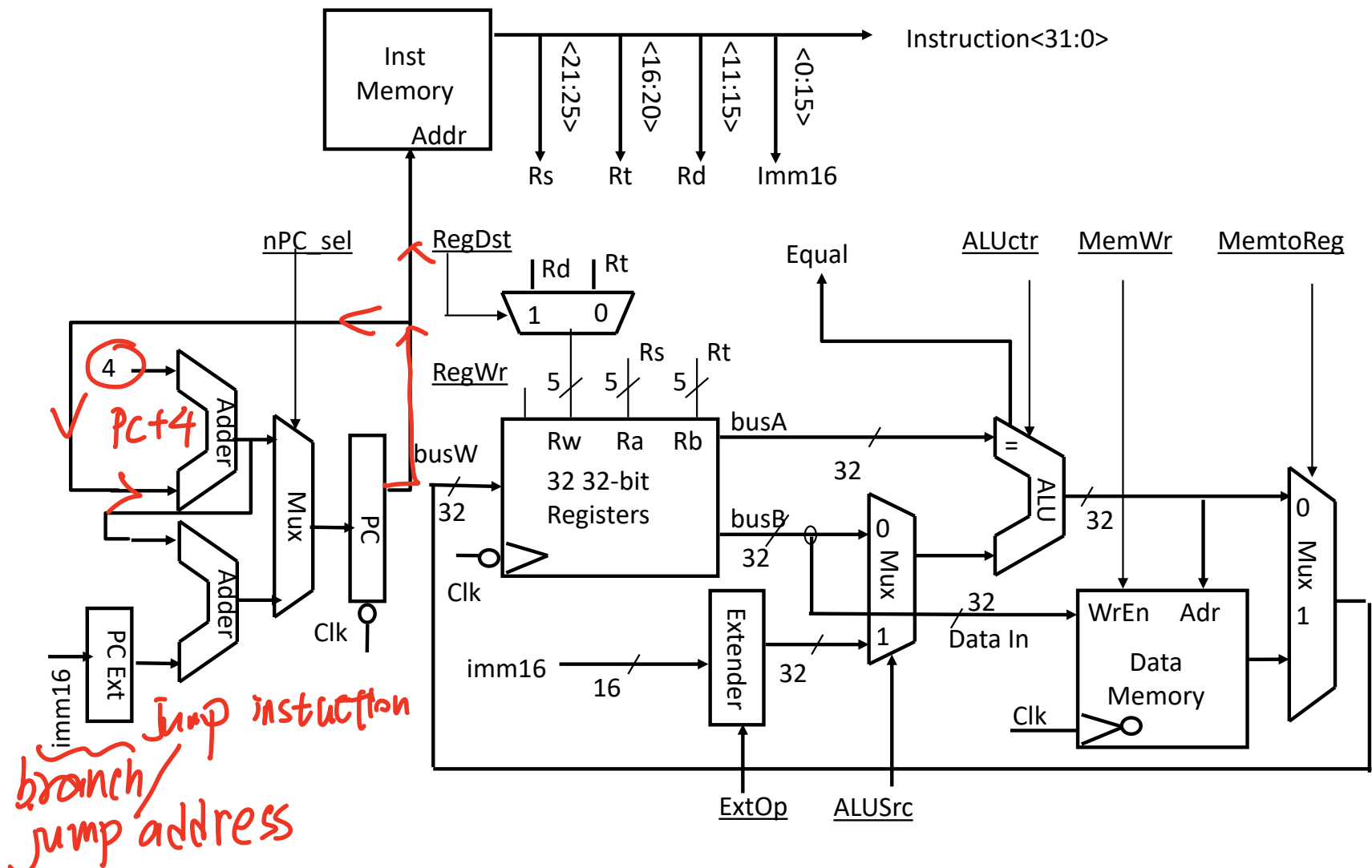
Single Cycle Datapath – Arithmetic/Logic



$R[rd] = R[rs] \text{ op } R[rt]$

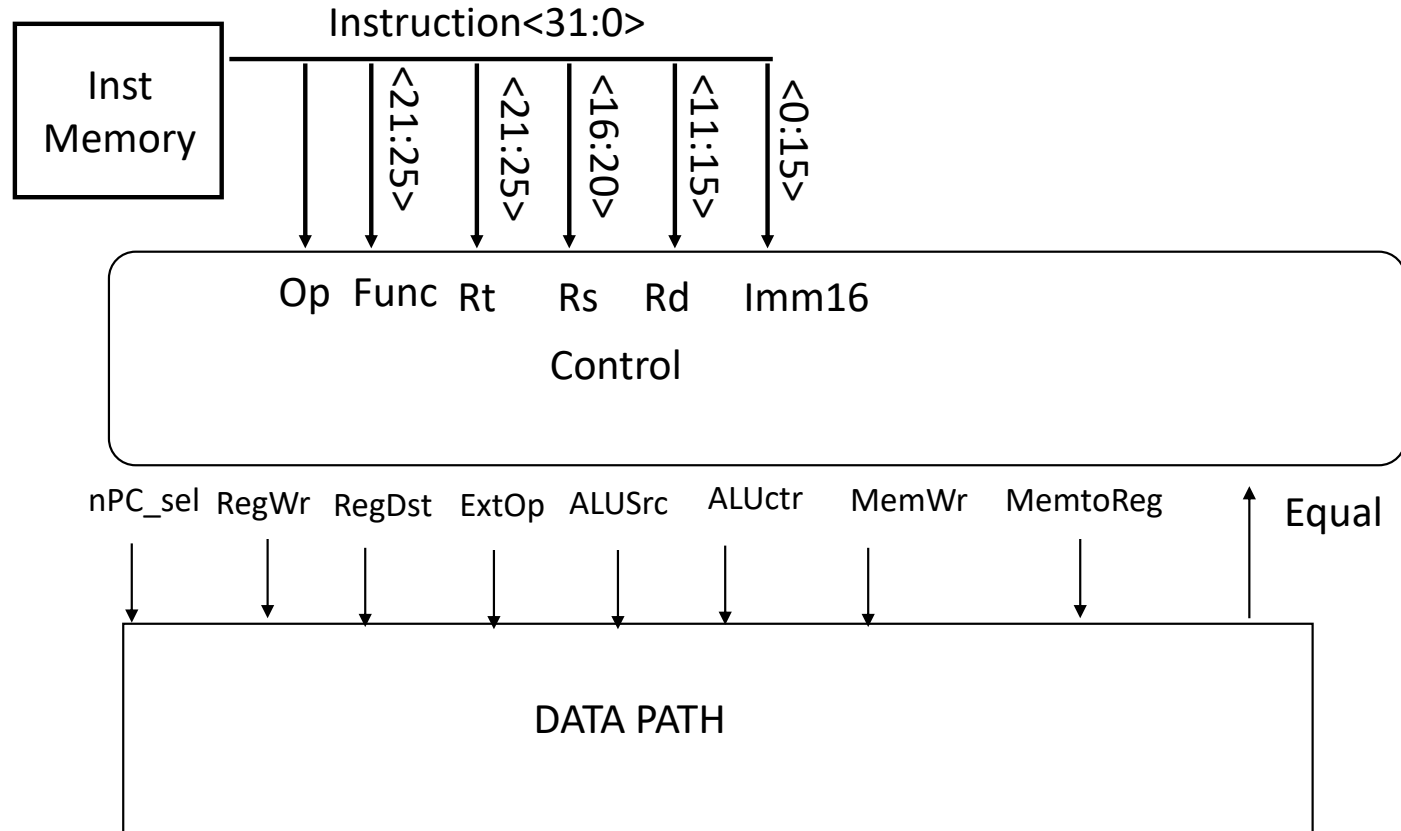
Rs, *Rt*, and *Rd* come from instruction's *rs*, *rt*, and *rd* fields

ALUctr and *RegWr*: control logic after decoding the instruction



Notes: $PC\ Ext(imm16) = SignExt(imm16) \times 4$. Rt is the destination for arithmetic ops with immediate.

Generating Control Signals



The Five Steps on Instruction Execution

- Fetch the instruction from the instruction memory
- Register fetch and instruction decode
- ALU execution or memory address calculation (using ALU)
- Read the data from the data memory
- Write the data back to the register file

Not all instructions will require all the steps

Exercise

The MIPS pseudo-instruction *swap* exchanges the contents of two registers (e.g., `swap $s0, $s1`). After the sequence completes, the destination register has the original value of the source register, and the source register has the original value of the destination register.

This pseudo-instruction is synthesized by the MIPS assembler using three xor instructions:

```
xor a, a, b
```

```
xor b, a, b
```

```
xor a, a, b
```

An alternative ISA design for the MIPS might include an opcode for *swap*, thus making it a real instruction. Both the original and the alternative designs use a single cycle datapath. Describe the key modifications required in the datapath in order to implement *swap*.

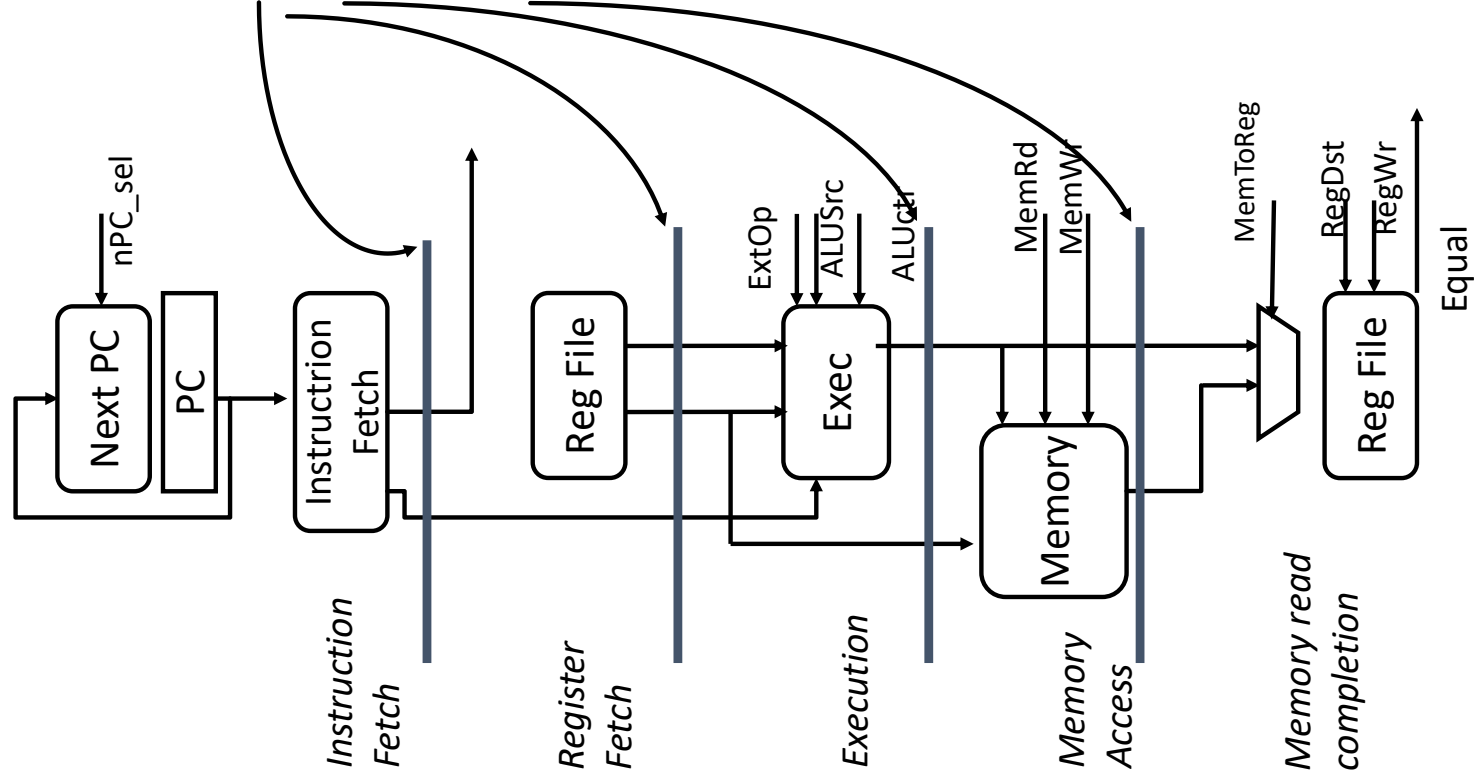
Discussion

- Cycle time for load is much longer than needed for all other instructions
- Since we have a single cycle, all operations take the longest cycle time, i.e., the cycle time of load
 - Instruction Memory Access Time
 - Register File Access Time
 - ALU Delay (address calculation)
 - Data Memory Access Time
 - Register File Setup Time

Multiple Cycle Datapath

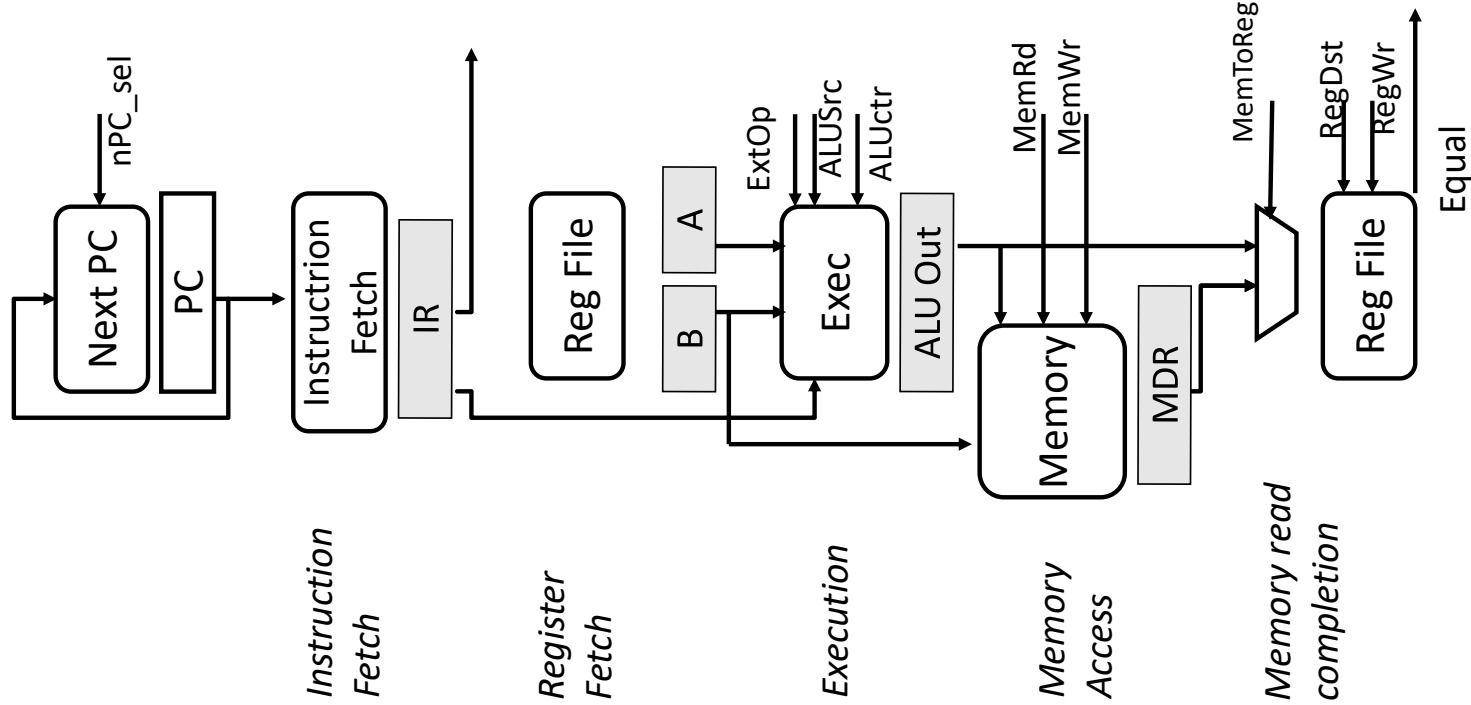
- Break each instruction into a series of steps corresponding to the functional unit operations required
 - Each of these steps will take 1 clock cycle
- We can use a functional unit more than once per instruction as long as it is used on different clock cycles
 - Less resource use; savings on real estate
- We can use a unified memory – no need to have separate instruction & data memories

Single Cycle Datapath – Logical Blocks



Add registers between blocks for storing intermediate values/results

Multiple Cycle Datapath – Logical Blocks



Add registers between blocks for storing intermediate values/results. The physical data transfers between these registers realize the logical instructions.

R-Type Add

- Logical register transfers:
 - $R[rd] \leftarrow R[rs] + R[rt]; PC \leftarrow PC + 4$
- Physical register transfers:
 - $IR \leftarrow MEM[PC]; PC \leftarrow PC + 4$ ① *instruction fetch*
 - $A \leftarrow R[rs]; B \leftarrow R[rt]$ ② *instruction decode & register fetch*
 - $ALUOut \leftarrow A + B$ ③ *execution, memory address computation, or branch completion*
 - $R[rd] \leftarrow ALUOut$ ④ *memory access, or R-Type instruction completion*

Load

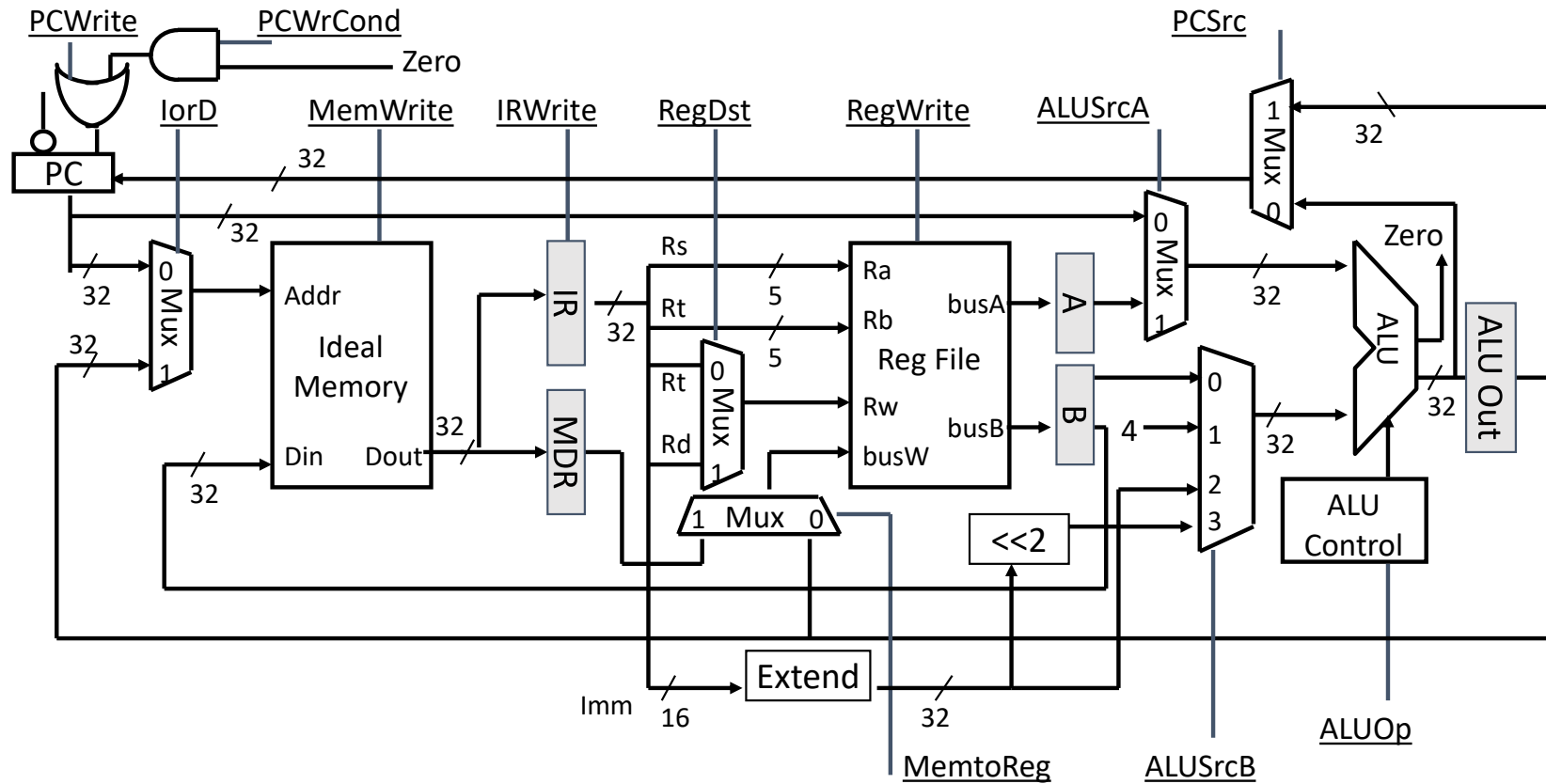
- Logical register transfers:
 - $R[rt] \leftarrow \text{Mem}[R[rs] + \text{SignExt}(\text{Imm16})]; PC \leftarrow PC + 4$
- Physical register transfers:
 - $IR \leftarrow \text{MEM}[PC]; PC \leftarrow PC + 4$ ① *instruction fetch*
 - $A \leftarrow R[rs]$ ② *instruction decode & register fetch*
 - $\text{ALUOut} \leftarrow A + \text{SignExt}(\text{Imm16})$ ③ *execution, memory address computation, or branch completion*
 - $\text{MDR} \leftarrow \text{Mem}[\text{ALUOut}]$ ④ *memory access, or R-Type instruction completion)*
 - $R[rt] \leftarrow \text{MDR}$ ⑤ *memory read completion*

Store

- Logical register transfers:
 - $\text{Mem}[\text{R}[\text{rs}] + \text{SignExt}(\text{Imm16})] \leftarrow \text{R}[\text{rt}]; \text{PC} \leftarrow \text{PC} + 4$
- Physical register transfers:
 - $\text{IR} \leftarrow \text{MEM}[\text{PC}]; \text{PC} \leftarrow \text{PC} + 4$ ① *instruction fetch*
 - $\text{A} \leftarrow \text{R}[\text{rs}]; \text{B} \leftarrow \text{R}[\text{rt}]$ ② *instruction decode & register fetch*
 - $\text{ALUOut} \leftarrow \text{A} + \text{SignExt}(\text{Imm16})$ ③ *execution, memory address computation, or branch completion*
 - $\text{Mem}[\text{ALUOut}] \leftarrow \text{B}$ ④ *memory access, or R-Type instruction completion*

Branch: BEQ

- Logical register transfers:
 - $PC \leftarrow (A == B) ? (PC + 4 + \text{SignExt}(\text{Imm16}) \times 4) : (PC + 4)$
- Physical register transfers:
 - $IR \leftarrow \text{MEM}[PC]; PC \leftarrow PC + 4$ ① *instruction fetch*
 - $A \leftarrow R[\text{rs}]; B \leftarrow R[\text{rt}]; \text{ALUOut} \leftarrow PC + \text{SignExt}(\text{Imm16}) \times 4$ ② *instruction decode & register fetch*
 - If ($A == B$) $PC \leftarrow \text{ALUOut}$ ③ *execution, memory address computation, or branch completion*



Generating Control Signals

- Generating control signals is more complex than for the single cycle datapath.
- Control signals are generated using a finite state machine or using microprogramming.
 - Refer to the textbook mentioned in the last slide if interested.

Single Cycle Datapath

- If all instructions must complete within one clock cycle, then the cycle time has to be large enough to accommodate the *slowest* instruction.
- For example, lw \$t0, -4(\$sp) needs 8ns/13ns, assuming the delays shown here.

	Case 1	Case 2
reading the instruction memory	2ns	3ns
reading the base register \$sp	1ns	2ns
computing memory address \$sp-4	2ns	3ns
reading the data memory	2ns	3ns
storing data back to \$t0	1ns	2ns

Single Cycle Datapath

- If we make the cycle time 8ns then *every* instruction will take 8ns, even if they don't need that much time.
- For example, the instruction add \$s4, \$t1, \$t2 really needs just 6ns/10ns.

	Case 1	Case 2
reading the instruction memory	2ns	3ns
reading registers \$t1 and \$t2	1ns	2ns
computing \$t1 + \$t2	2ns	3ns
storing the result into \$s0	1ns	2ns

Multiple Cycle Datapath

- The GNU C compiler, gcc, is reported to have the following instruction mix

Instruction	Frequency
Arithmetic	48%
Loads	22%
Stores	11%
Branches	19%

- What is the performance gain of a multiple cycle implementation over a single cycle implementation for running gcc?

Further Reading

- Computer Organization and Design – The Hardware Software Interface RISC-V edition, 2nd edition (2020)

¿?

Pipelining

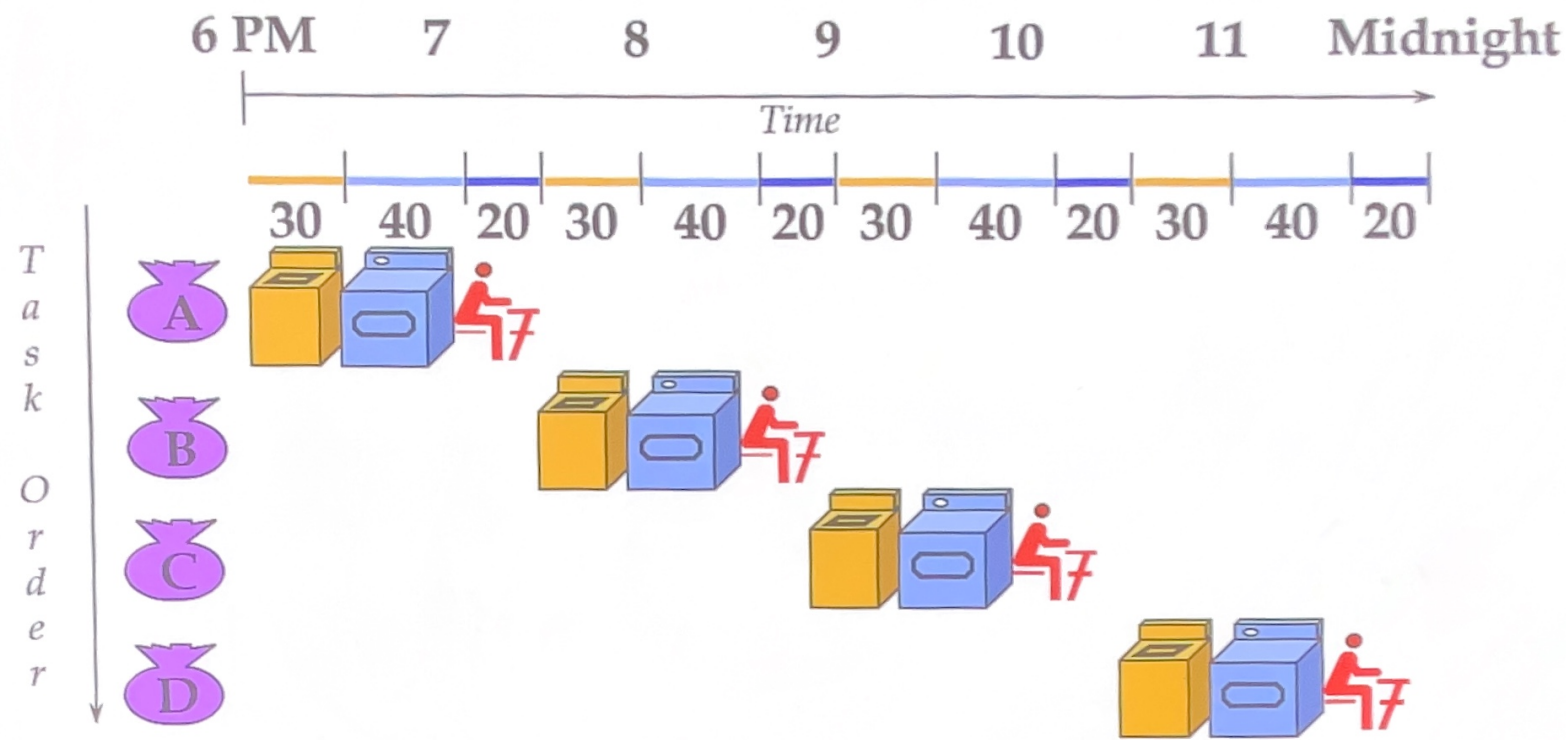
mano@cs.auckland.ac.nz

Pipelining

- Pipelining provides an efficient way of executing multiple tasks concurrently
- The laundry example
 - We have four loads of laundry to do. This consists of washing, drying, and folding.
 - Washer takes 30 minutes
 - Dryer takes 40 minutes
 - Folding takes 20 minutes

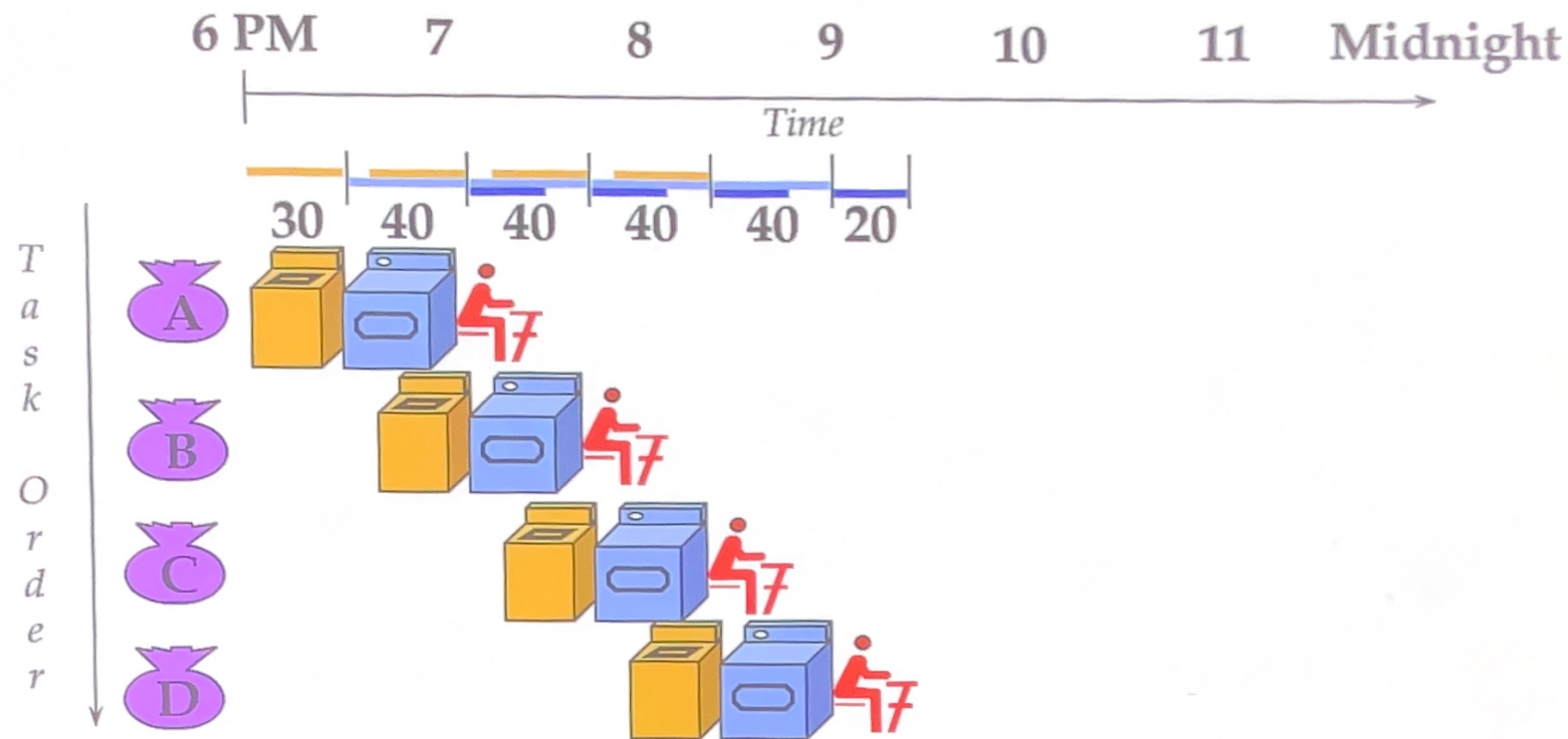


Sequential Laundry



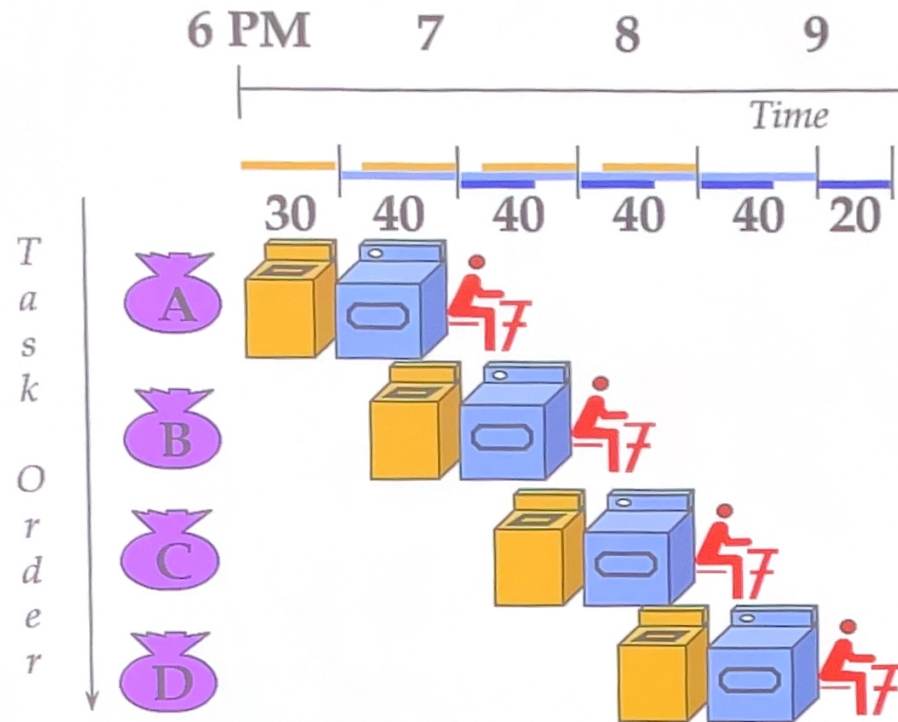
Sequential laundry takes 6 hours for 4 loads

Pipelined Laundry



Pipelined laundry takes 3 ½ hours for 4 loads

Pipelining



Multiple tasks operating simultaneously

Pipelining doesn't help latency of single task, it helps *throughput* of entire workload

Pipeline rate limited by the slowest pipeline stage

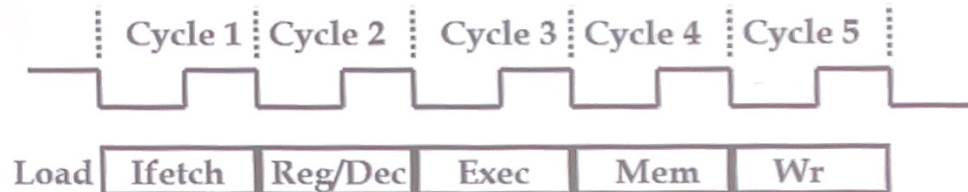
Potential speedup = number pipe stages

Unbalanced lengths of pipe stages reduces speedup

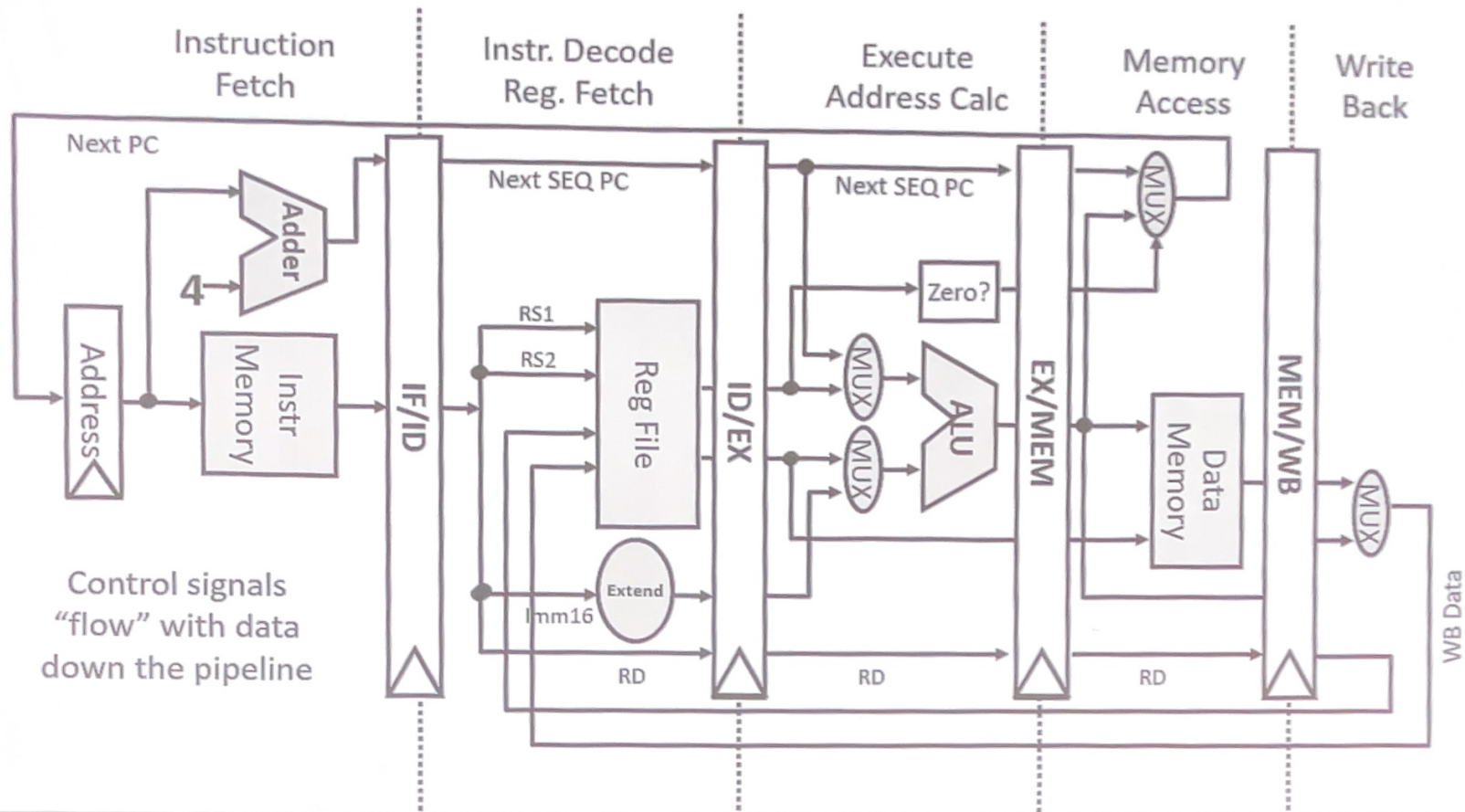
Also, need time to "fill" and "drain" the pipeline.

The Load Instruction

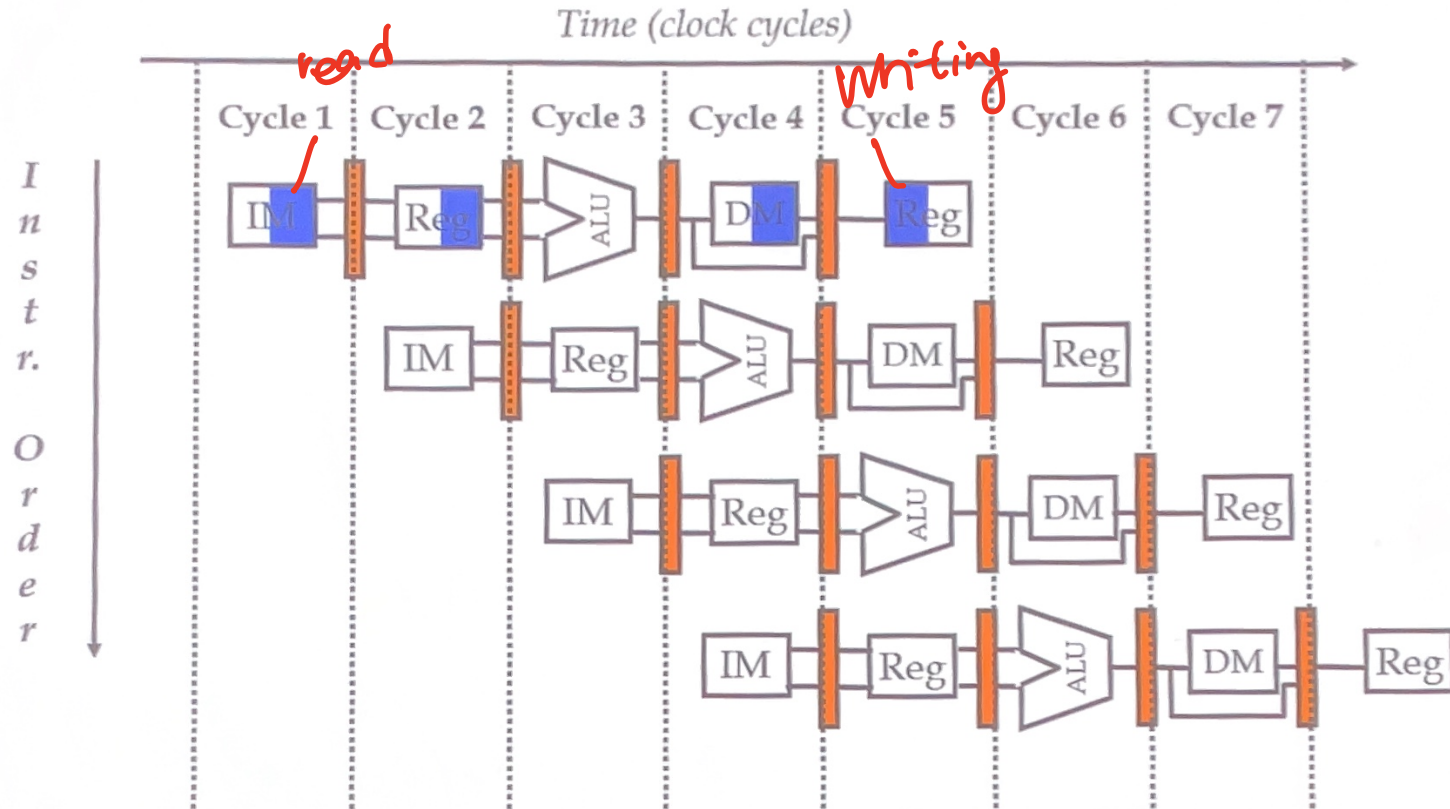
- Fetch the instruction from the instruction memory
- Register fetch and instruction decode
- Calculate the memory address (using ALU)
- Read the data from the data memory
- Write the data back to the register file



The Steps of the MIPS Datapath

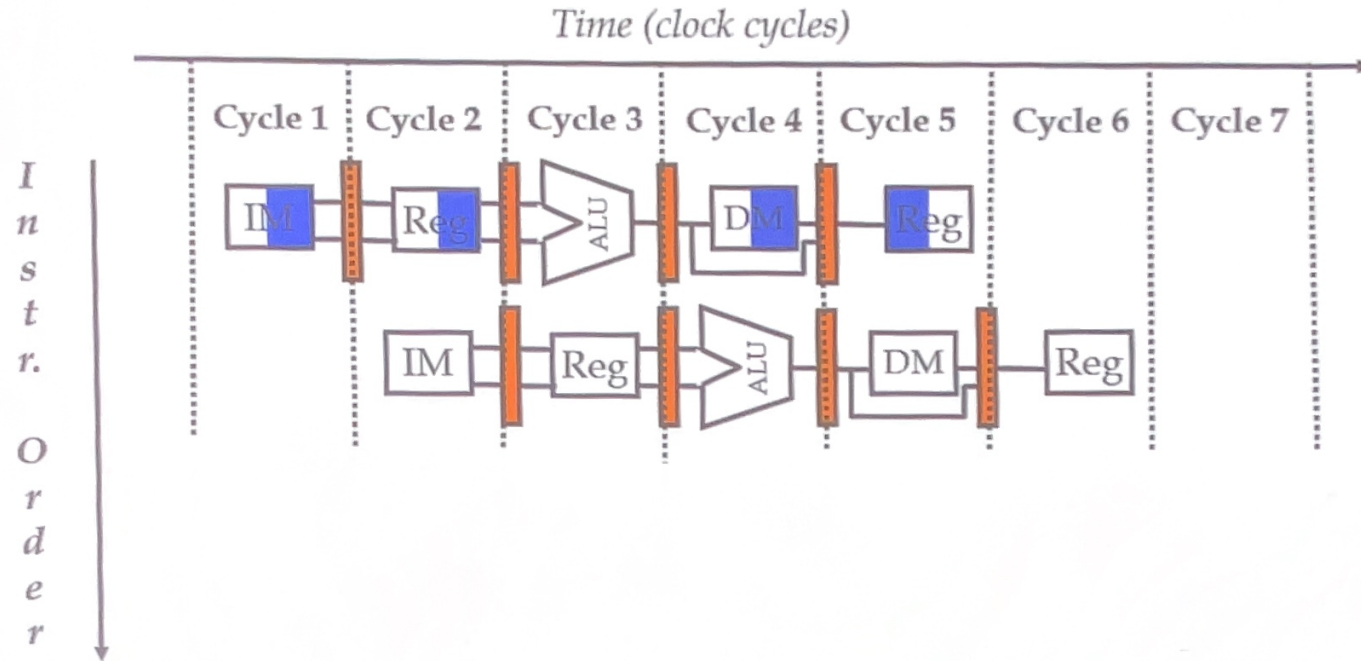


Visualizing Pipelining



Multiple instructions are in various stages at the same time

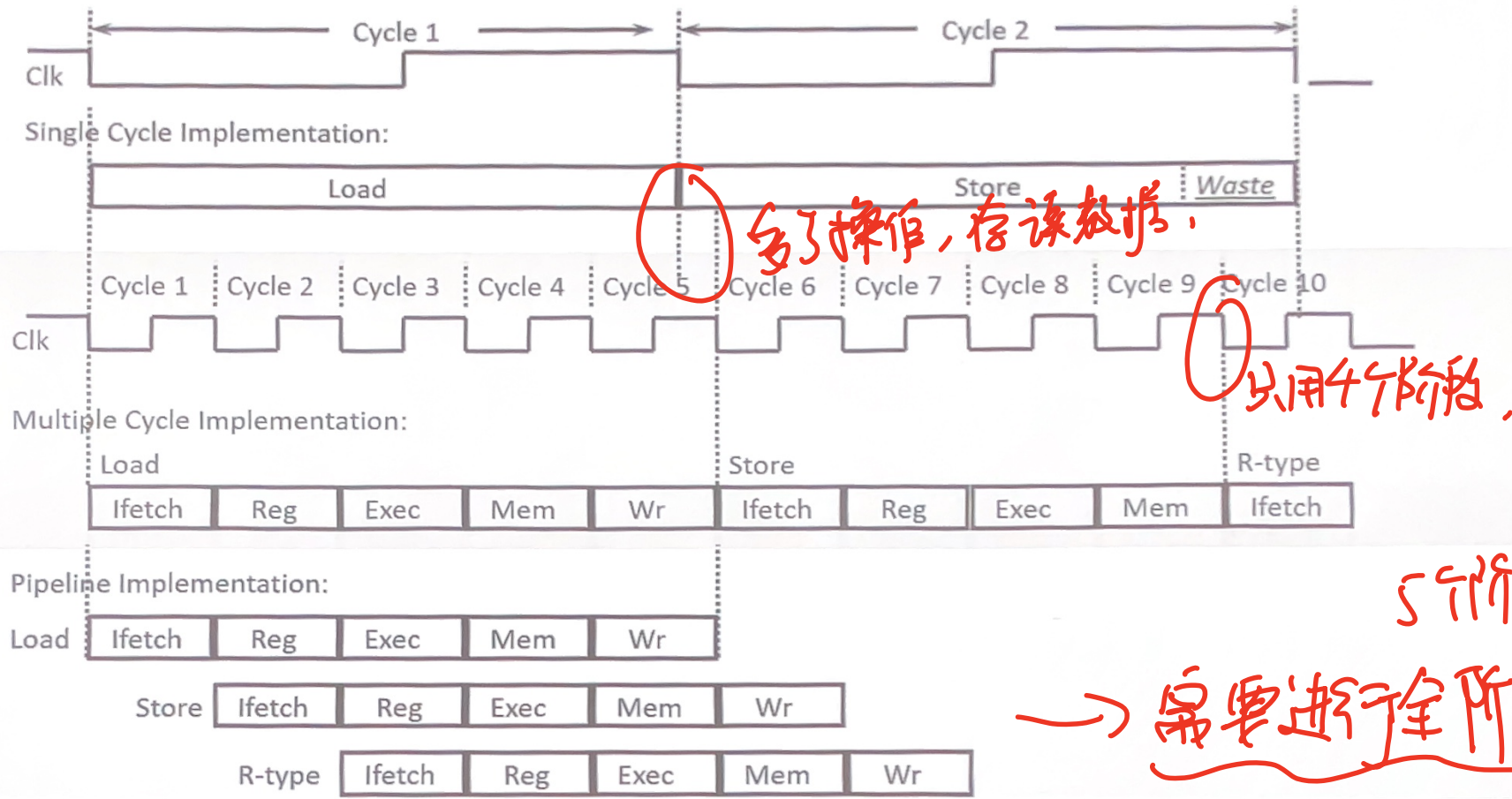
Visualizing Pipelining



Can help with answering questions such as:

1. How many cycles does it take to execute this code?
2. What is the ALU doing during cycle 4?
3. Are two instructions trying to use the same resource at the same time?

Comparison of Processor Design



5个阶段即使
不需要
也要进行全阶段
→ 需要进行全阶段
2.5倍

Comparison of Processor Design

- Suppose we have 100 instructions to execute
 - Single cycle processor has a cycle time of 45 ns
 - Multi cycle and pipeline processors have cycle times of 10 ns
 - The multi cycle processor has a CPI of 3.8 时钟周期数 $\leftarrow$ 平均指令数
- Single cycle processor
 - $45 \text{ ns/cycle} \times 1 \text{ CPI} \times 100 \text{ inst} = 4500 \text{ ns}$
- Multi cycle processor
 - $10 \text{ ns/cycle} \times 3.8 \text{ CPI} \times 100 \text{ inst} = 3800 \text{ ns}$
- Ideal pipelined machine
 - $10 \text{ ns/cycle} \times (1 \text{ CPI} \times 100 \text{ inst} + 4 \text{ cycle "drain"}) = 1040 \text{ ns}$
- Ideal pipelined vs. single cycle speedup
 - $4500 \text{ ns} / 1040 \text{ ns} = 4.33$

Pipeline Hazards 险事

- A hazard prohibits the execution of a planned instruction in the proper cycle.
- Three forms of hazards may exist in a pipeline.
 - Structural hazard
 - Data hazard
 - Control hazard

Structural Hazards

- Structural hazards arise when attempting to use the same resource at the same time.
 - If we have a combined dryer/washer, then we won't be able to use the dryer and washer simultaneously.
 - If we have a unified memory that holds both data as well as instructions, then at cycle 4 when there is data access, we won't be able to read an instruction out.

Structural hazard occurs when a planned instruction cannot execute in the proper clock cycle because the hardware cannot support the combination of instructions that are set to execute in the given clock cycle.